Yara - Rilevazione di file con malware

Description

Introduzione

YARA è uno strumento di pattern matching usato principalmente per rilevare e classificare malware, basandosi su firme definite dall'utente. È molto utilizzato in cybersecurity per analizzare file sospetti, eseguibili e persino la memoria di un sistema alla ricerca di minacce informatiche.

A cosa serve?

- **Rilevazione di malware**: Identifica file malevoli basandosi su stringhe di testo, pattern esadecimali o espressioni regolari.
- **Threat Hunting**: Usato dagli analisti di sicurezza per cercare indicatori di compromissione (IoC) nei sistemi.
- **Analisi forense**: Aiuta a identificare componenti malevoli in file o processi.
- **Reverse Engineering**: Può essere usato per cercare firme specifiche all'interno del codice binario di un programma.

Avvio rapido

Per scrivere una regola base e utilizzarla per scansionare un file:

```
echo "rule dummy { condition: true }" > my_first_rule
yara my_first_rule my_first_rule
```

Se tutto va a buon fine, l'output sarà:

```
dummy my_first_rule
```

Scrivere Regole YARA

Struttura base

La regola più semplice possibile è la seguente:

```
rule dummy {
    condition: false
}
```

Ogni regola in YARA inizia con la parola chiave `rule`, seguita da un identificatore conforme alle convenzioni di C.

Identificatori riservati

Non usare i seguenti identificatori:

```
contains, condition, ascii, any, and, all, in, global, for, fullword, filesize,
false, entrypoint, int16be, int8be, int32, int16, int8, include, import,
of, or, not, nocase, meta, matches, int32be, uint16, uint8, true, them,
strings, rule, private, wide, uint32be, uint16be, uint8be
```

Definizione di regole

Le regole generalmente hanno due sezioni principali:

- **Definizione delle stringhe**
- **Condizioni**

Esempio:

```
rule ExampleRule {
    strings:
        $my_text_string = "text here"
        $my_hex_string = { E2 34 A1 C8 23 FB }
    condition:
```

```
        $my_text_string or $my_hex_string
}
```

Commenti

Supporta commenti in stile C:

```
/* Questo è un commento multi-linea */
rule CommentExample // Questo è un commento su una linea
{
    condition: false // Regola di esempio
}
```

Tipi di Stringhe in YARA

YARA supporta tre tipi di stringhe:

- Stringhe esadecimali
- Stringhe di testo
- Espressioni regolari

Stringhe Esadecimali

Wildcards (caratteri jolly)

I wildcards sono operatori a livello di nibble, il che significa che puoi definire solo una parte del byte e lasciare l'altra sconosciuta.

```
rule WildcardExample
{
    strings:
        $hex_string = { E2 34 ?? C8 A? FB }
    condition:
        $hex_string
}
```

Jumps (salti)

Permettono di definire stringhe con porzioni di contenuto e lunghezza variabili.

```
rule JumpExample
{
    strings:
        $hex_string = { F4 23 [4-6] 62 B4 }
    condition:
        $hex_string
}
```

Le seguenti stringhe corrisponderanno al pattern:

- F4 23 01 02 03 04 62 B4
- F4 23 00 00 00 00 00 62 B4
- F4 23 15 82 A3 04 45 22 62 B4

Ogni jump [X-Y] deve soddisfare la condizione 0 <= X <= Y

Esempi di jump validi:

```
FE 39 45 [0-8] 89 00
FE 39 45 [23-45] 89 00
FE 39 45 [1000-2000] 89 00
```

Esempio non valido:

```
FE 39 45 [10-7] 89 00
```

Se i limiti inferiore e superiore sono uguali, è possibile scrivere un singolo numero tra parentesi:

```
FE 39 45 [6] 89 00
```

La stringa sopra è equivalente a entrambe queste:

```
FE 39 45 [6-6] 89 00
```

```
FE 39 45 ?? ?? ?? ?? ?? ?? 89 00
```

Da YARA 2.0 è possibile utilizzare jump illimitati:

```
FE 39 45 [10-] 89 00
FE 39 45 [-] 89 00
```

Alternatives (alternative)

Permettono di fornire diverse alternative per un dato frammento.

```
rule AlternativesExample1
{
    strings:
        $hex_string = { F4 23 ( 62 B4 | 56 ) 45 }
    condition:
        $hex_string
}
```

Questa regola corrisponderà a qualsiasi file contenente F42362B445 o F4235645.

Non ci sono limiti alla quantità di sequenze alternative che puoi fornire, né alle loro lunghezze:

```
rule AlternativesExample2
{
    strings:
        $hex_string = { F4 23 ( 62 B4 | 56 | 45 ?? 67 ) 45 }
    condition:
        $hex_string
}
```

Stringhe di Testo

```
rule TextExample
{
    strings:
        $text_string = "foobar"
    condition:
```

```
        $text_string
}
```

Stringhe Case-insensitive

```
rule CaseInsensitveTextExample
{
    strings:
        $text_string = "foobar" nocase
    condition:
        $text_string
}
```

Stringhe Wide-character

Il modificatore "wide" può essere utilizzato per cercare stringhe codificate con due byte per carattere, tipico in molti file eseguibili binari.

```
rule WideCharTextExample
{
    strings:
        $wide_string = "Borland" wide
    condition:
        $wide_string
}
```

Ricerca di parole intere

Il modificatore "fullword" garantisce che la stringa corrisponderà se appare circondata da caratteri non alfanumerici.

La stringa "domain", se definita come fullword, non corrisponderà a "www.mydomain.com" ma corrisponderà a "www.my-domain.com" e "www.domain.com".

Espressioni Regolari

```
rule RegExpExample1
{
    strings:
        $re1 = /md5: [0-9a-zA-Z]{32}/
```

```
        $re2 = /state: (on|off)/
    condition:
        $re1 and $re2
}
```

Le espressioni regolari di YARA riconoscono i seguenti metacaratteri:

- `.` - Qualsiasi carattere
- `^` - Inizio della stringa
- `# Tecniche Avanzate YARA

Stringhe in YARA

1. Uso di variabili in YARA

YARA supporta l'uso di variabili per rendere le regole più dinamiche e facilmente adattabili. Ecco un esempio:

```
rule DynamicExample {
    strings:
        $text_string = "malware"
    condition:
        $text_string and filesize < 1MB
}
```

In questo esempio, la variabile `$text_string` rappresenta una stringa che può essere facilmente modificata per altri pattern senza riscrivere l'intera regola.

2. Rilevamento di varianti di malware

YARA può essere utilizzato per identificare varianti di un malware con piccole modifiche nelle firme. Ad esempio:

```
rule VariantExample {
    strings:
        $pattern1 = "malicious_code"
        $pattern2 = { D1 23 A4 56 }
    condition:
```

```
        $pattern1 or $pattern2
}
```

Questa regola rileva malware che può variare tra una stringa testuale e una sequenza esadecimale.

3. Crittografia delle stringhe

Un malware può criptare le sue stringhe per evitare che vengano rilevate da YARA. Un esempio di come YARA può essere utilizzato per rilevare una stringa criptata:

```
rule EncryptedStringExample {
    strings:
        $encrypted_string = { E1 D4 A8 C9 45 67 }
    condition:
        for any of ($encrypted_string*) : (true)
}
```

Questa regola cerca una stringa criptata specifica, anche se potrebbe sembrare "casuale".

4. Modulo PE per rilevare file eseguibili Windows

Il modulo PE di YARA è utile per l'analisi dei file eseguibili di Windows (PE - Portable Executable). Ecco un esempio di come rilevare il numero di sezioni di un file PE:

```
import "pe"

rule PESectionCount {
    condition:
        pe.number_of_sections > 5
}
```

Questa regola rileva i file PE che hanno più di cinque sezioni.

5. Modulo Math per operazioni complesse

Il modulo Math permette di eseguire calcoli avanzati. Ecco come usarlo:

```
import "math"

rule MathExample {
    condition:
        math.pow(2, 3) == 8
}
```

Questo esempio usa il modulo math per calcolare la potenza e confrontarla con il valore di 8.

6. Integrazione con Suricata

YARA può essere utilizzato in integrazione con strumenti come Suricata per automatizzare il rilevamento di malware. Un esempio di configurazione per eseguire YARA con Suricata potrebbe essere il seguente:

```
suricata -r my_traffic.pcap --yara-rules my_rules.yara
```

In questo caso, YARA viene eseguito su un file di traffico di rete (.pcap) per rilevare eventuali minacce.

7. Ottimizzazione delle regole per ridurre i falsi positivi

Per ridurre i falsi positivi, è possibile combinare condizioni più specifiche:

```
rule OptimizedExample {
    strings:
        $a = "malicious"
    condition:
        $a and filesize < 500KB and pe.entrypoint > 0x1000
}
```

Questa regola ottimizza la ricerca includendo più condizioni, riducendo così il numero di falsi positivi.

8. Uso di YARA per Incident Response

Durante un incidente di sicurezza, YARA può essere usato per esaminare file sospetti. Ecco un esempio di regola per identificare file compromessi:

```
rule SuspiciousFile {
    strings:
        $suspicious_string = "compromised_data"
    condition:
        $suspicious_string and filesize > 1MB
}
```

- Fine della stringa
- `|` - Alternativa
- `()` - Raggruppamento
- `[]` - Classe di caratteri

Le seguenti quantificazioni sono riconosciute:

- - Match 0 o più volte
- `+` - Match 1 o più volte
- `?` - Match 0 o 1 volta
- `{n}` - Match esattamente n volte
- `{n,}` - Match almeno n volte
- `{,m}` - Match 0 a m volte
- `{n,m}` - Match da n a m volte

Variante non greedy

Tutte queste quantificazioni hanno una variante non greedy, seguita da un punto interrogativo (?):

- `?` - Match 0 o più volte, non greedy
- `+?` - Match 1 o più volte, non greedy
- `??` - Match 0 o 1 volta, non greedy
- `{n}?` - Match esattamente n volte, non greedy

- `{n,}?` - Match almeno n volte, non greedy
- `{,m}?` - Match 0 a m volte, non greedy
- `{n,m}?` - Match da n a m volte, non greedy

Il matching non greedy, noto anche come lazy matching, cerca di far corrispondere il minor testo possibile. Ad esempio, l'espressione regolare `.*` è greedy e corrisponderà all'intera stringa di input, mentre l'espressione regolare `.*?` è non greedy e corrisponderà solo al primo carattere della stringa di input.

Escape

Le seguenti sequenze di escape sono riconosciute:

- `\t` - Tab (HT, TAB)
- `\n` - Nuova riga (LF, NL)
- `\r` - Ritorno a capo (CR)
- `\f` - Form feed (FF)
- `\a` - Campanello d'allarme
- `\xNN` - Carattere il cui numero ordinale è il numero esadecimale dato

Classi di caratteri

- `\w` - Corrisponde a un carattere di parola (alfanumerico più "_")
- `\W` - Corrisponde a un carattere non di parola
- `\s` - Corrisponde a un carattere di spazio vuoto
- `\S` - Corrisponde a un carattere non di spazio vuoto
- `\d` - Corrisponde a una cifra decimale
- `\D` - Corrisponde a un carattere non cifra

Condizioni in YARA

Le condizioni possono contenere:

- Operatori booleani tipici: `and`, `or` e `not`

- Operatori relazionali: `>=`, `<=`, `<`, `>`, `==` e `!=`
- Operatori aritmetici (`+`, , , `\`, `%`) e operatori bit a bit (`&`, `|`, `<<`, `>>`, `~`, `^`)

```
rule Example
{
    strings:
        $a = "text1"
        $b = "text2"
        $c = "text3"
        $d = "text4"
    condition:
        ($a or $b) and ($c or $d)
}
```

Conteggio delle Stringhe

A volte abbiamo bisogno di sapere non solo se una certa stringa è presente, ma quante volte appare nel file o nella memoria del processo.

```
rule CountExample
{
    strings:
        $a = "dummy1"
        $b = "dummy2"
    condition:
        #a == 6 and #b > 10
}
```

Offset delle Stringhe o Indirizzi Virtuali

A volte dobbiamo sapere se la stringa si trova a un offset specifico nel file o a un indirizzo virtuale all'interno dello spazio di indirizzo del processo.

```
rule AtExample
{
    strings:
        $a = "dummy1"
        $b = "dummy2"
    condition:
        $a at 100 and $b at 200
}
```

L'operatore `in` consente di cercare la stringa all'interno di un intervallo di offset o indirizzi.

```
rule InExample
{
    strings:
        $a = "dummy1"
        $b = "dummy2"
    condition:
        $a in (0..100) and $b in (100..filesize)
}
```

Puoi anche ottenere l'offset o l'indirizzo virtuale della i-esima occorrenza della stringa $a usando `@a[i]`.

Dimensione del File

```
rule FileSizeExample
{
    condition:
        filesize > 200KB
}
```

Entry Point Eseguibile

```
rule EntryPointExample1 {
    strings:
        $a = { E8 00 00 00 00 }
    condition:
        $a at entrypoint
}

rule EntryPointExample2 {
    strings:
        $a = { 9C 50 66 A1 ?? ?? ?? 00 66 A9 ?? ?? 58 0F 85 }
    condition:
        $a in (entrypoint..entrypoint + 10)
}
```

Accesso ai Dati in una Posizione Specifica

Condizioni che dipendono dai dati memorizzati in un determinato offset del file o indirizzo virtuale della memoria:

- `int8(<offset o indirizzo virtuale>)`
- `int16(<offset o indirizzo virtuale>)`
- `int32(<offset o indirizzo virtuale>)`
- `uint8(<offset o indirizzo virtuale>)`
- `uint16(<offset o indirizzo virtuale>)`
- `uint32(<offset o indirizzo virtuale>)`
- `int8be(<offset o indirizzo virtuale>)`
- `int16be(<offset o indirizzo virtuale>)`
- `int32be(<offset o indirizzo virtuale>)`
- `uint8be(<offset o indirizzo virtuale>)`
- `uint16be(<offset o indirizzo virtuale>)`
- `uint32be(<offset o indirizzo virtuale>)`

Il parametro `<offset o indirizzo virtuale>` può essere qualsiasi espressione che restituisca un intero senza segno, incluso il valore restituito da una delle funzioni uintXX stesse. Come esempio, vediamo una regola per distinguere i file PE:

```
rule IsPE {
    condition:
        // Firma MZ all'offset 0 e ...
        uint16(0) == 0x5A4D and
        // ... firma PE all'offset memorizzato nell'header MZ a 0x3C
        uint32(uint32(0x3C)) == 0x00004550
}
```

Set di Stringhe

È possibile esprimere che il file dovrebbe contenere un certo numero di stringhe da un determinato set.

```
rule OfExample1 {
    strings:
        $a = "dummy1"
        $b = "dummy2"
        $c = "dummy3"
    condition:
        2 of ($a,$b,$c)
}

rule OfExample2 {
    strings:
        $foo1 = "foo1"
        $foo2 = "foo2"
        $foo3 = "foo3"
    condition:
        2 of ($foo*) /* equivalente a 2 of ($foo1,$foo2,$foo3) */
}

rule OfExample3 {
    strings:
        $foo1 = "foo1"
        $foo2 = "foo2"
        $bar1 = "bar1"
        $bar2 = "bar2"
    condition:
        3 of ($foo*,$bar1,$bar2)
}
```

È possibile utilizzare `($*)` per fare riferimento a tutte le stringhe nella regola, o scrivere la parola chiave equivalente `them` per una maggiore leggibilità.

```
rule OfExample4 {
    strings:
        $a = "dummy1"
        $b = "dummy2"
        $c = "dummy3"
    condition:
        1 of them /* equivalente a 1 of ($*) */
}
```

In tutti gli esempi precedenti, il numero di stringhe è stato specificato da una costante numerica, ma è possibile utilizzare qualsiasi espressione che restituisca un valore numerico. Anche le parole chiave `any` e `all` possono essere utilizzate:

- `all of them` /* tutte le stringhe nella regola */

- `any of them` /* qualsiasi stringa nella regola */
- `all of ($a*)` /* tutte le stringhe il cui identificatore inizia con $a */
- `any of ($a,$b,$c)` /* qualsiasi tra $a, $b o $c */
- `1 of ($*)` /* uguale a "any of them" */

Applicare la Stessa Condizione a Molte Stringhe

- `for <espressione> of <set_stringhe>: (<espressione_booleana>)`
- `for any of ($a,$b,$c): ($ at entrypoint)`
- `any of ($a,$b,$c)` equivale a `for any of ($a,$b,$c): ($)`
- `for all of them: (# > 3)`
- `for all of ($a*): (@ > @b)`

Riferimento ad Altre Regole

```
rule Rule1 {
    strings:
        $a = "dummy1"
    condition:
        $a
}

rule Rule2 {
    strings:
        $a = "dummy2"
    condition:
        $a and Rule1
}
```

Altre Caratteristiche delle Regole

Regole Globali

Le regole globali offrono la possibilità di imporre restrizioni a tutte le regole contemporaneamente.

```
global rule SizeLimit {
    condition:
        filesize < 2MB
}
```

Regole Private

```
private rule PrivateRuleExample { ... }
```

Tag delle Regole

I tag possono essere utilizzati successivamente per filtrare l'output di YARA e mostrare solo le regole che ti interessano.

```
rule TagsExample1 : Foo Bar Baz { ... }
rule TagsExample2 : Bar { ... }
```

Metadati

```
rule MetadataExample {
    meta:
        my_identifier_1 = "Some string data"
        my_identifier_2 = 24
        my_identifier_3 = true
    strings:
        $my_text_string = "text here"
        $my_hex_string = { E2 34 A1 C8 23 FB }
    condition:
        $my_text_string or $my_hex_string
}
```

Operatori e condizioni

Operatori logici

```
$a or $b
$a and $b
```

```
not $a
```

Operatori relazionali e aritmetici

```
>=, <=, <, >, ==, !=, +, -, *, /, %, & (AND bitwise), | (OR b
itwise), <<, >>, ~ (NOT bitwise), ^ (XOR)
```

Contare le occorrenze

```
rule CountExample {
    strings:
        $a = "dummy1"
        $b = "dummy2"
    condition:
        #a == 6 and #b > 10
}
```

Offset e indirizzi virtuali

```
rule AtExample {
    strings:
        $a = "dummy1"
        $b = "dummy2"
    condition:
        $a at 100 and $b at 200
}
```

Uso dell'operatore `in`:

```
rule InExample {
    strings:
        $a = "dummy1"
        $b = "dummy2"
```

```
    condition:
        $a in (0..100) and $b in (100..filesize)
}
```

Dimensione del file

```
rule FileSizeExample {
    condition:
        filesize > 200KB
}
```

Identificazione del punto di ingresso di un eseguibile

```
rule EntryPointExample {
    strings:
        $a = { E8 00 00 00 00 }
    condition:
        $a at entrypoint
}
```

Accesso ai dati in posizioni specifiche

```
rule IsPE {
    condition:
        uint16(0) == 0x5A4D and uint32(uint32(0x3C)) == 0x000
04550
}
```

Regole con insiemi di stringhe

```
rule OfExample {
    strings:
```

```
        $a = "dummy1"
        $b = "dummy2"
        $c = "dummy3"
    condition:
        2 of ($a,$b,$c)
}
```

Riferimenti a altre regole

```
rule Rule1 {
    strings:
        $a = "dummy1"
    condition:
        $a
}
rule Rule2 {
    strings:
        $a = "dummy2"
    condition:
        $a and Rule1
}
```

Regole Globali e Private

```
global rule SizeLimit {
    condition:
        filesize < 2MB
}
private rule PrivateRuleExample {
    condition:
        false
}
```

Tag e Metadata

```
rule TagsExample : Foo Bar Baz {
    condition: true
}
rule MetadataExample {
    meta:
        author = "John Doe"
        version = 1.0
    condition:
        true
}
```

Moduli

YARA supporta moduli scritti in C per condizioni avanzate. Alcuni moduli comuni:

- `PE` (analisi eseguibili Windows)
- `Math` (calcoli matematici)
- `Strings` (manipolazione avanzata di stringhe)

Per usarli:

```
import "pe"
rule CheckPE {
    condition:
        pe.number_of_sections > 5
}
```

Tecniche Avanzate

1. Uso di variabili in YARA

YARA supporta l'uso di variabili per rendere le regole più dinamiche e facilmente adattabili.
Ecco un esempio:

```
rule DynamicExample {
    strings:
        $text_string = "malware"
    condition:
        $text_string and filesize < 1MB
}
```

In questo esempio, la variabile `$text_string` rappresenta una stringa che può essere facilmente modificata per altri pattern senza riscrivere l'intera regola.

2. Rilevamento di varianti di malware

YARA può essere utilizzato per identificare varianti di un malware con piccole modifiche nelle firme. Ad esempio:

```
rule VariantExample {
    strings:
        $pattern1 = "malicious_code"
        $pattern2 = { D1 23 A4 56 }
    condition:
        $pattern1 or $pattern2
}
```

Questa regola rileva malware che può variare tra una stringa testuale e una sequenza esadecimale.

3. Crittografia delle stringhe

Un malware può criptare le sue stringhe per evitare che vengano rilevate da YARA.
Un esempio di come YARA può essere utilizzato per rilevare una stringa criptata:

```
rule EncryptedStringExample {
    strings:
```

```
        $encrypted_string = { E1 D4 A8 C9 45 67 }
    condition:
        for any of ($encrypted_string*) : (true)
}
```

Questa regola cerca una stringa criptata specifica, anche se potrebbe sembrare "casuale".

4. Modulo PE per rilevare file eseguibili Windows

Il modulo PE di YARA è utile per l'analisi dei file eseguibili di Windows (PE - Portable Executable). Ecco un esempio di come rilevare il numero di sezioni di un file PE:

```
import "pe"

rule PESectionCount {
    condition:
        pe.number_of_sections > 5
}
```

Questa regola rileva i file PE che hanno più di cinque sezioni.

5. Modulo Math per operazioni complesse

Il modulo Math permette di eseguire calcoli avanzati. Ecco come usarlo:

```
import "math"

rule MathExample {
    condition:
        math.pow(2, 3) == 8
}
```

Questo esempio usa il modulo math per calcolare la potenza e confrontarla con il valore di 8.

6. Integrazione con Suricata

YARA può essere utilizzato in integrazione con strumenti come Suricata per automatizzare il rilevamento di malware. Un esempio di configurazione per eseguire YARA con Suricata potrebbe essere il seguente:

```
suricata -r my_traffic.pcap --yara-rules my_rules.yara
```

In questo caso, YARA viene eseguito su un file di traffico di rete (`.pcap`) per rilevare eventuali minacce.

7. Ottimizzazione delle regole per ridurre i falsi positivi

Per ridurre i falsi positivi, è possibile combinare condizioni più specifiche:

```
rule OptimizedExample {
    strings:
        $a = "malicious"
    condition:
        $a and filesize < 500KB and pe.entrypoint > 0x1000
}
```

Questa regola ottimizza la ricerca includendo più condizioni, riducendo così il numero di falsi positivi.

8. Uso di YARA per Incident Response

Durante un incidente di sicurezza, YARA può essere usato per esaminare file sospetti.
Ecco un esempio di regola per identificare file compromessi:

```
rule SuspiciousFile {
    strings:
        $suspicious_string = "compromised_data"
    condition:
        $suspicious_string and filesize > 1MB
}
```

Questa regola cerca la stringa "compromised_data" in file di dimensioni superiori a 1MB.